

**Using Chest X-Ray Image Classification to Differentiate Between COVID-19, Pneumonia,
and Healthy Lungs**

Landon Nguyen

Department of Data Science, University of Texas Arlington

DATA 4380: Data Problems

Dr. Rostami, Masoud

December 8, 2025

Introduction

The events of the COVID-19 global pandemic sparked interest in the development of tools that can quickly and efficiently identify different diseases. Said interest in combination with the rise of AI and deep learning models led to the profound idea of using image classification techniques to identify diseases via x-ray images. By focusing on chest x-rays that capture the state of the lungs, respiratory infections, especially COVID-19, can provide a basis for testing the predicative capabilities of Convolutional Neural Networks (CNN) in differentiating between healthy lungs and those infected by various diseases.

The use of predictive modeling to diagnose infections could provide two potential benefits moving forward. First, quick analysis of images enables faster and efficient diagnoses for numerous patients at one time. Second, predictive models may avoid major weaknesses of standard nucleic acid tests: sensitivity to disease progression and immune status. Furthermore, proof that predictive modeling can efficiently diagnose diseases could provide a basis for future integration of machine learning into healthcare practices, such as diagnosing other diseases, classifying case severity, etc. This paper discusses the use of image classification of chest x-rays to distinguish between healthy individuals, and those infected with COVID-19 or viral pneumonia.

Exploratory Data Analysis

To begin, the data was loaded using the Python library, TensorFlow (TF). The images were split between three classes (encoded numerically): COVID-19 (0), normal (1), and pneumonia (2).

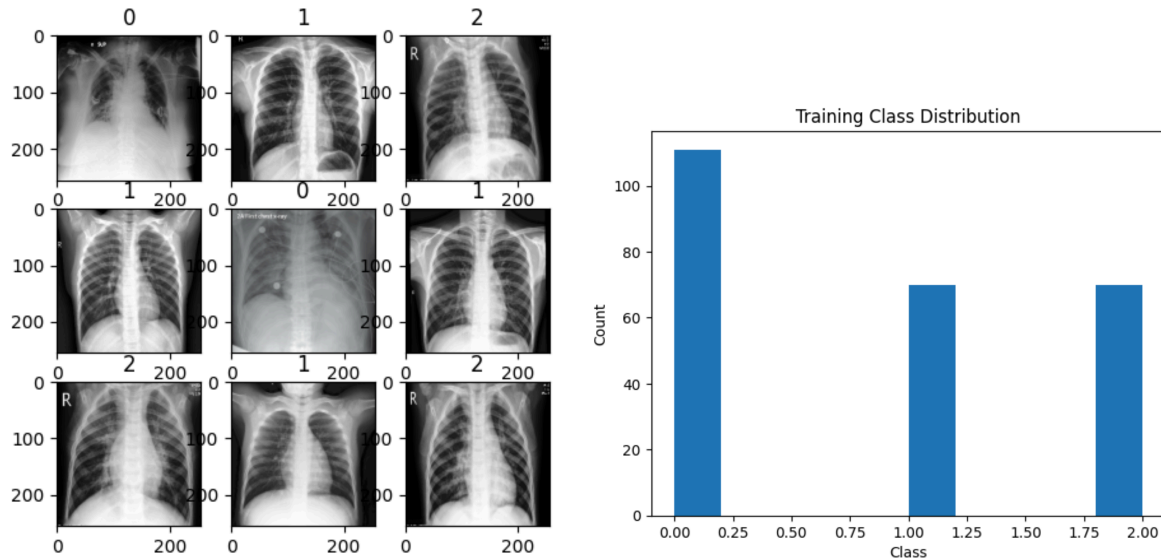


Figure 1. Sample images from the dataset and a distribution of class counts. (A) a sample subset of images. COVID-19 is encoded as 0; normal (health) lungs are encoded as 1; viral pneumonia is encoded as 2. (B) a distribution of class counts.

Figure 1 displays a subset of sample images from the dataset. From a glance, the COVID-19 images stand out quite clearly due to the cloudiness in the lungs. Normal and pneumonia images, however, can be quite difficult to distinguish from one another. Furthermore, a distribution of the classes demonstrates a slight imbalance, with COVID-19 being the majority class. The combination of imbalance and difficult distinguishability (between the minority classes) is a major problem for predictive models.

Before developing a methodology, two baseline models were created as a basis for tracking progression. For baseline modeling, a train/validation/test split of 80/10/10 was used. The first baseline was a standard artificial neural network (ANN; see Fig 2.)

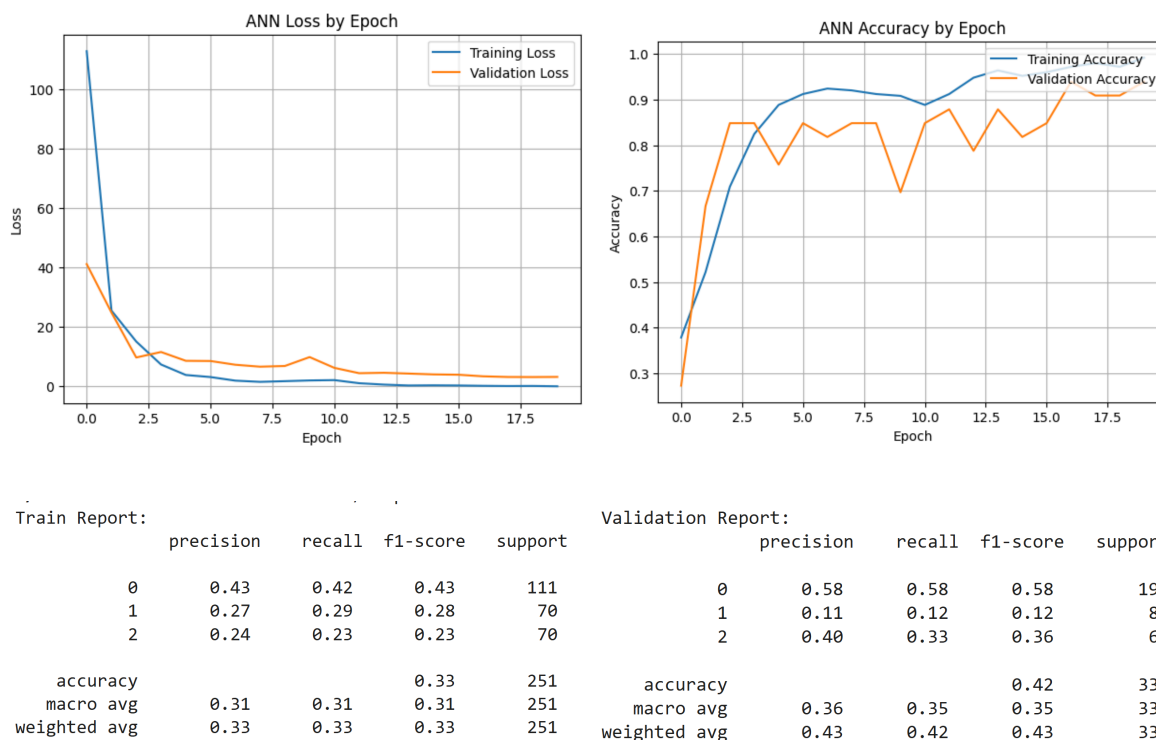


Figure 2. Baseline ANN model results. (A) A loss graph showing the loss progression of the model. (B) An accuracy graph showing the accuracy progression of the model. (C) A classification report of the model's performance on the training set. (D) A classification report of the model's performance on the validation set.

The results of the ANN model were quite underwhelming. The incredibly fast drop in loss from 100 to 0 implies that the model stopped learning very quickly. The accuracy plot shows that the model had inconsistent performance on the validation set and was overfitting to some extent. The classification reports show that the model was having a difficult time classifying the two minority classes, and an overall accuracy of 33% for training with 3 classes implies near-random predictions.

The second baseline attempted was a CNN model with one layer of convolution and pooling (see Fig. 3)

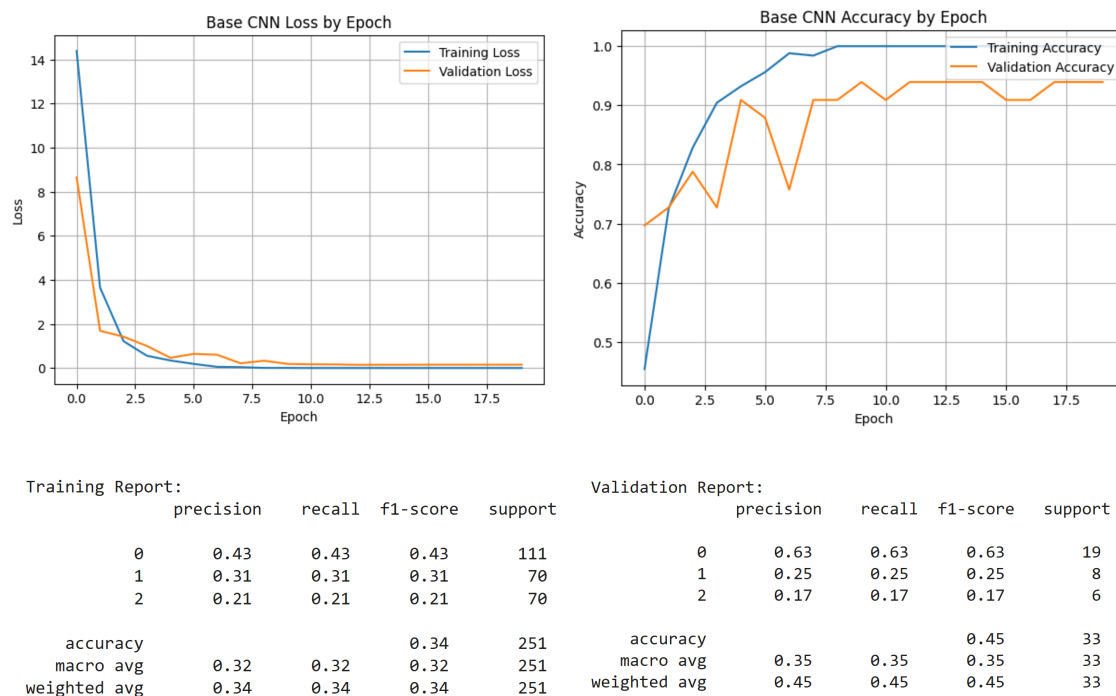


Figure 3. Baseline CNN model results. (A) A loss graph showing the loss progression of the model. (B) An accuracy graph showing the accuracy progression of the model. (C) A classification report of the model's performance on the training set. (D) A classification report of the model's performance on the validation set.

The baseline CNN's performance was nearly identical to the ANN's performance, with the exception of a lower starting loss (14 vs 100). That is, the model's predictions were nearly random; this implies that the model is not learning enough about the dataset.

Methodology

In order to maximize the predictive results of this study, two methodologies were tested to tackle three major issues with the dataset. The first challenge was to correct the imbalance issue in the dataset. The second challenge was to provide the model with more information about the classes, via data augmentation, to boost its learning capabilities. The third challenge,

discussed in more detail later, was that the dataset was too complex to build an effective model from scratch.

Method 1

Beginning with preprocessing, the first method combines random oversampling with Contrast Limited Adaptive Histogram Equalization (CLAHE) to tackle challenges one and two simultaneously. The idea is to balance the classes using random oversampling and apply contrast augmentation to the oversampled data to amplify the model's performance on the minority classes. Figure 4 displays the effects of these preprocessing steps

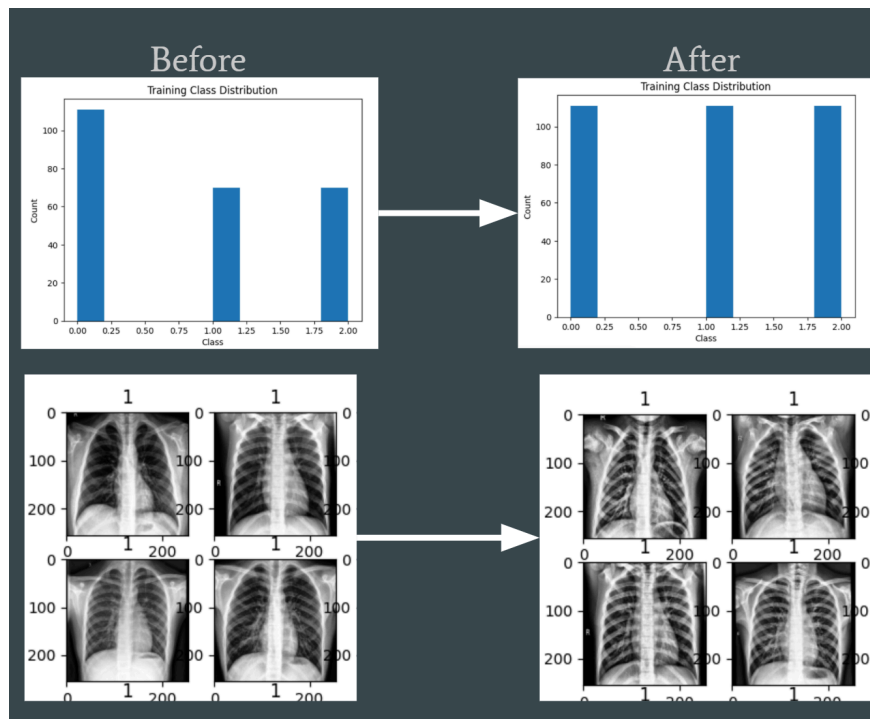


Figure 4. The effects of combining random oversampling with CLAHE. The before and after images compare the distribution of class counts before/after random oversampling, and the oversampled images from the normal class before/after CLAHE.

The oversampling technique balanced out the class counts. Furthermore, the CLAHE filter amplified the white areas inside the lungs of the x-ray images. One downside, however, is that the contrast filter also exaggerates the skeletal structure of the images.

Following the preprocessing stage, two modelling structures were employed. The first was a more developed CNN model built from scratch (see Fig. 5). The second was a transfer learning model, VGG16 (see Fig. 6).

Layer (type)	Output Shape	Param #
conv2d_3 (Conv2D)	(None, 256, 256, 16)	448
max_pooling2d_3 (MaxPooling2D)	(None, 128, 128, 16)	0
conv2d_4 (Conv2D)	(None, 128, 128, 32)	4,640
max_pooling2d_4 (MaxPooling2D)	(None, 64, 64, 32)	0
flatten_4 (Flatten)	(None, 131072)	0
dense_8 (Dense)	(None, 128)	16,777,344
dropout (Dropout)	(None, 128)	0
dense_9 (Dense)	(None, 64)	8,256
dropout_1 (Dropout)	(None, 64)	0
dense_10 (Dense)	(None, 3)	195

Total params: 16,790,883 (64.05 MB)
Trainable params: 16,790,883 (64.05 MB)
Non-trainable params: 0 (0.00 B)

Figure 5. Overview of the CNN model used for method 1. The model is a TF sequential model that includes two layers of convolution and pooling, and dropout of 0.5.

The CNN model is slightly more developed than the baseline model, including an extra layer of convolution and pooling as well as a 50% dropout to control overfitting. The model was trained with a learning rate of 0.001, 20 epochs, and a batch size of 32.

Layer (type)	Output Shape	Param #
vgg16 (Functional)	(None, 8, 8, 512)	14,714,688
flatten_2 (Flatten)	(None, 32768)	0
dense_4 (Dense)	(None, 128)	4,194,432
dense_5 (Dense)	(None, 3)	387

Total params: 18,909,507 (72.13 MB)
 Trainable params: 4,194,819 (16.00 MB)
 Non-trainable params: 14,714,688 (56.13 MB)

Figure 6. An overview of the VGG16 model used for method 1. The model is a TF sequential that starts with a frozen VGG16 transfer learning model using imagenet as its initial weights. The VGG16 model employs transfer learning to increase the number of convolutional layers without dramatically increasing training time. The model was trained with a learning rate of 0.001, 10 epochs, and a batch size of 32.

Method 2

The second method attempted differs in both preprocessing and modeling. Image augmentation including random horizontal flipping, sheer, zoom, rotation, and height/width shifting was applied to each image upon loading the dataset. Furthermore, instead of oversampling, class weights are used to counteract the imbalance issue. The focal point of method 2 was the DenseNet201 model employed (see Fig. 7).

Layer (type)	Output Shape	Param #	Connected to
input_layer_4 (InputLayer)	(None, 224, 224, 3)	0	-
zero_padding2d_6 (ZeroPadding2D)	(None, 230, 230, 3)	0	input_layer_4[0]...
conv1_conv (Conv2D)	(None, 112, 112, 64)	9,408	zero_padding2d_6...
conv1_bn (BatchNormalizatio...	(None, 112, 112, 64)	256	conv1_conv[0][0]
conv1_relu (Activation)	(None, 112, 112, 64)	0	conv1_bn[0][0]

bn (BatchNormalizatio...	(None, 7, 7, 1920)	7,680	conv5_block32_co...
relu (Activation)	(None, 7, 7, 1920)	0	bn[0][0]
global_average_poo... (GlobalAveragePool...	(None, 1920)	0	relu[0][0]
dropout_2 (Dropout)	(None, 1920)	0	global_average_p...
dense_2 (Dense)	(None, 3)	5,763	dropout_2[0][0]

Total params: 18,327,747 (69.91 MB)
 Trainable params: 5,763 (22.51 KB)
 Non-trainable params: 18,321,984 (69.89 MB)

Figure 7. An overview of the DenseNet201 model used for method 2. The model was a TF functional API that starts with DenseNet201 using imagenet for its initial weights and includes a dropout of 0.3.

The main strategy with the DenseNet201 model was to unfreeze the last 43 convolutional layers, which would allow the model to learn more about the dataset used. The strategy was employed to overcome two challenges. First, the dataset was too complex for a small number of layers to learn enough about the data to effectively distinguish the classes. Second, imagenet is not designed to provide effective weights for x-ray images. The model was trained with a learning rate of 1.0×10^{-5} , 25 epochs, and a batch size of 16.

Results

Method 1

Figure 8 displays the results of the CNN model from method 1.

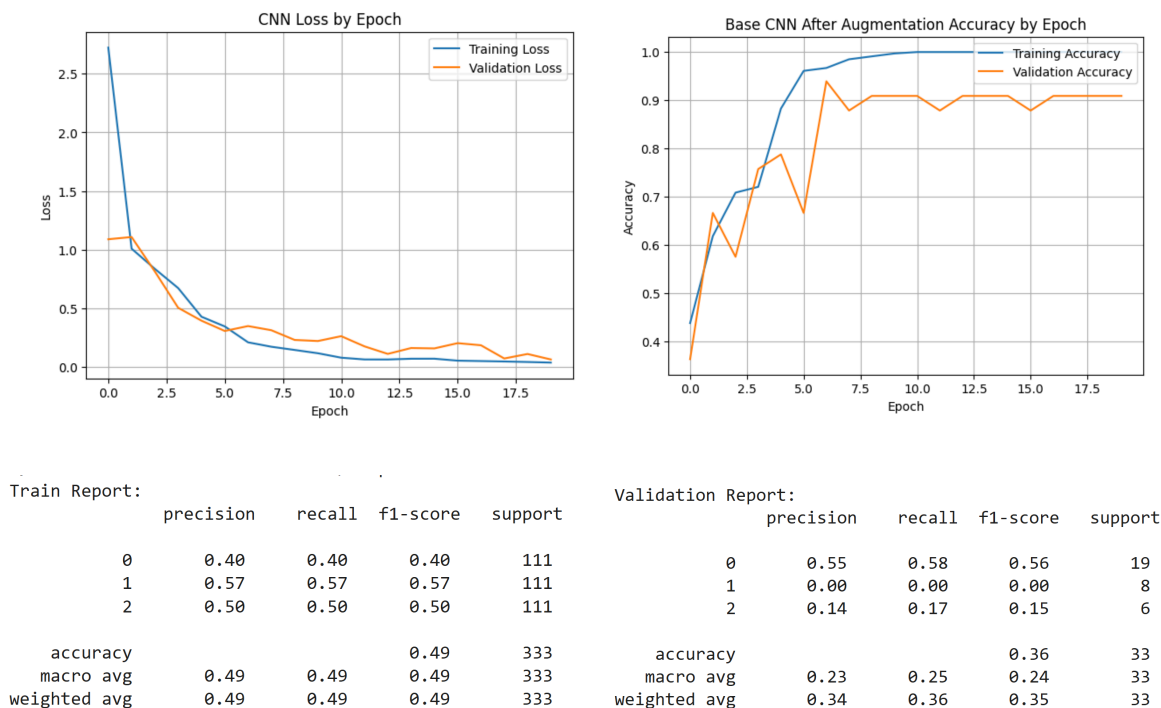
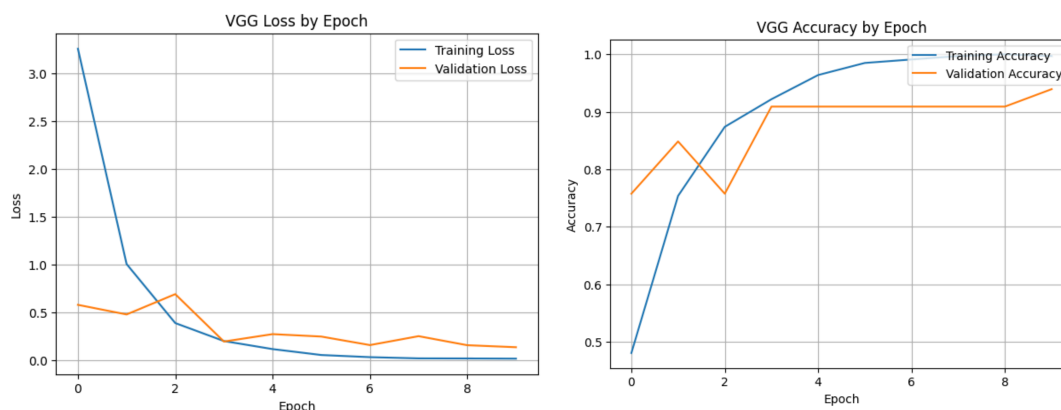


Figure 8. CNN results following method 1. (A) A loss graph showing the loss progression of the model. (B) An accuracy graph showing the accuracy progression of the model. (C) A classification report of the model's performance on the training set. (D) A classification report of the model's performance on the validation set.

Compared to the baseline, the CNN model shows good progression in terms of loss. The starting loss was significantly lower (3 vs 100) and the loss curve was a little smoother. In terms of accuracy, however, the model was still overfitting despite the inclusion of dropout. The training metrics were far better with balanced performance on each class and a macro F1 score of 0.49. However, the validation metrics were far from desirable. In fact, the model could not classify the normal class (1).

Figure 9 displays the results of the VGG16 model following method 1.



Train Report:					Validation Report:				
	precision	recall	f1-score	support		precision	recall	f1-score	support
0	0.46	0.46	0.46	111	0	0.68	0.68	0.68	19
1	0.52	0.52	0.52	111	1	0.50	0.38	0.43	8
2	0.52	0.52	0.52	111	2	0.25	0.33	0.29	6
accuracy			0.50	333	accuracy			0.55	33
macro avg	0.50	0.50	0.50	333	macro avg	0.48	0.46	0.47	33
weighted avg	0.50	0.50	0.50	333	weighted avg	0.56	0.55	0.55	33

Figure 9. VGG16 results following method 1. (A) A loss graph showing the loss progression of the model. (B) An accuracy graph showing the accuracy progression of the model. (C) A classification report of the model's performance on the training set. (D) A classification report of the model's performance on the validation set.

The VGG16 model performed significantly better than the baseline model. The loss graph is smoother and the overfitting in accuracy was less extreme by the end of the training process. The train and validation metrics are fairly balanced with macro F1 scores of 0.5 and 0.47, respectively. However, the validation classification report shows that the model is still struggling to classify the two minority classes.

Method 2

Figure 10 displays the results of the DenseNet201 model following method 2.

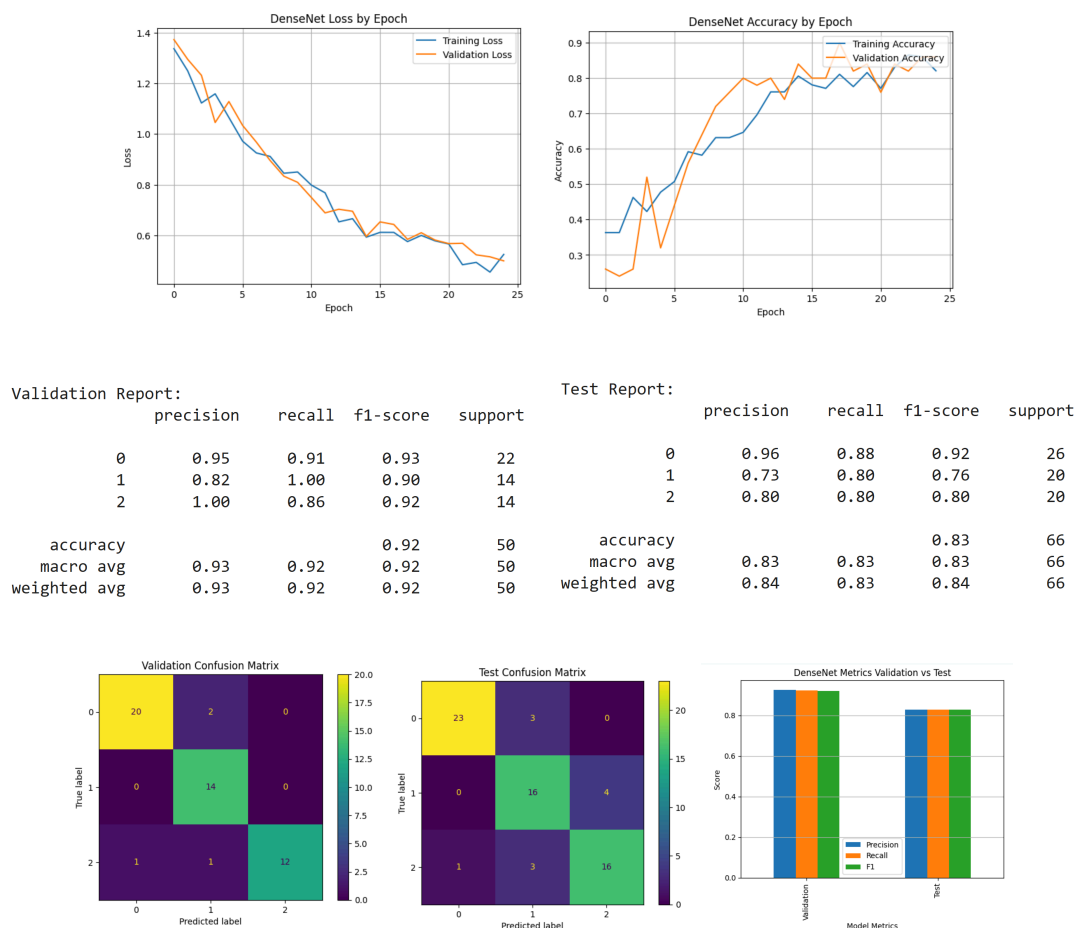


Figure 10. DenseNet201 results following method 2. (A) A loss graph showing the loss progression of the model. (B) An accuracy graph showing the accuracy progression of the model. (C) A classification report of the model's performance on the validation set. (D) A classification report of the model's performance on the test set. (E) Validation confusion matrix. (F) Test confusion matrix. (G) Visual comparison of the validation and test metrics.

The DenseNet201 model was remarkably better than both the baseline models and the method 1 models. The training progression seen in the loss and accuracy graphs was far less dramatic, meaning that the model was effectively learning throughout the entire training process. Furthermore, the accuracy plot shows very little issue with overfitting. The classification report shows that the model can make good predictions for each class and that the performance for each

class is mostly balanced. However, the model does still perform the best on the majority class. The confusion matrix demonstrates that the model's biggest weakness is differentiating between the normal and pneumonia classes. The model reached a macro F1 score of 0.92 on the validation set and 0.83 on the test set which implies some variance. In order to properly understand the extent of this variance, the model was used to predict on the validation set 30 times, and the macro F1 scores from each set of predictions was distributed (see Fig. 11).

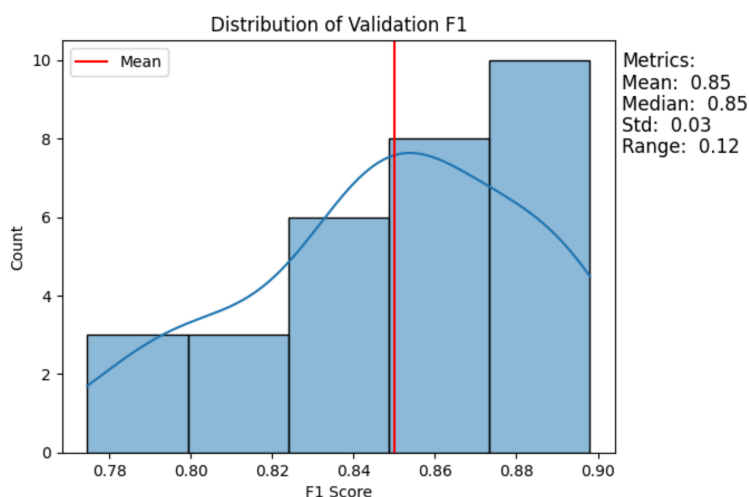


Figure 11. Distribution of validation F1 scores from the DenseNet201 model's predictions. The mean and median were both 0.85; the standard deviation was 0.03; the range was 0.12. Despite the mean and median F1 scores both being 0.85, the distribution still appears to be skewed. This is likely due to the sensitivity of F1 to class imbalance. The most notable metric, however, is the range of 0.12 and standard deviation of 0.3. The range implies that the model's performance on different datasets could vary to a great extent. Furthermore, based on the standard deviation under a standard normal assumption, the F1 score could be expected to change by about 0.06 for 95% of future predictions.

Conclusion

The DenseNet201 model was, by far, the best performing model. Unfreezing the layers of a heavy transfer learning model appears to be the most effective strategy for classifying x-ray images between COVID-19, normal, and viral pneumonia. However, the variance of the DenseNet201 model is still concerning. Future work should emphasize reducing the variance of the model to get more precise and consistent results. One such method could be using a stronger transfer learning model. As mentioned before, imagenet lacks specificity towards x-ray images. Other studies in this domain have had much better results using transfer learning models made specifically for x-ray imaging and COVID-19.

In conclusion, deep learning image classification models have remarkable potential for diagnosing diseases and streamlining healthcare practices. Ultimately, as of now, the model could prove to be a useful tool for classifying patients as potentially having COVID-19 or potentially not. The model still lacks the performance to work as a standalone test, but could be an effective precedent when used in conjunction with standard nucleic acid testing.